Occupational Health & Safety Remote

(OHS-Remote)

1.1 Team:

Rafael Soares Cardoso: Team Leader, Developer

Responsibilities: Act as Scrum Master, planning ahead on the next steps for the project using the Scrum Method to create sprints. Lead the group on through the project requirements while working alongside them to develop it.

James Konieczntt: Developer

Responsibilities: Work with the team through the development of the project, developing all the requirements following the plan set for each sprint individually.

Urgen Lama: Developer

Responsibilities: Work with the team through the development of the project, developing all the requirements following the plan set for each sprint individually.

1.2 Sponsor/Client: Jennifer Murray (jmurray5077@gmail.com)

This document has the purpose of specifying all the project requirements for OHS Report, such as the technologies used to develop it, team members, project specifications and features

1.3 Scope of the System:

OHS Remote has the objective of generating Safe Job Procedures manuals/handbooks based on their NAICS Codes inserted by the customers. The SJP document shall be generated using OpenAI's generative AI after the payment is completed.

1.4 Definitions, Acronyms, and Abbreviations:

- SJP: Safe Job Procedure
- OHS: Occupational Health & Safety (System's given name)
- NAICS: North American Industry Classification System
- AI: Artificial Intelligence

1.5 Reference Material:

- North American Industry Classification System (NAICS)
- Text generation | OpenAI API
- Workplace health and safety | ontario.ca

1.6 Overview of the Document

The remainder of this document is organized into the following key sections to provide a structured roadmap of the system's design and requirements:

- **Section 2: Specific Requirements**

Contains four comprehensive, prioritized matrix tables detailing the core constraints and operations of the system:

  - *Functional Requirements:* What the system must explicitly do.
  - *Non-Functional Requirements:* Performance metrics, security standards, and quality attributes.
  - *External Interface Requirements:* How the software interacts with hardware, users, and external software.
  - *Technical Requirements:* Development rules, execution environments, and architectural constraints.

- **Section 3: Tech Stack**

Outlines the specific technologies selected for the front-end, back-end, frameworks, and database layers, along with a technical justification for each choice.

- **Section 4: Project Management Software**

Details the administrative and collaborative environment of the team, defining the tools used for communication, documentation, task management, diagramming, version control, and development (IDEs).

- **Section 5: Appendices**

Provides supplementary reference materials to support implementation, including a comprehensive Glossary, Analysis Models, Supporting Information, and formal References.

2. Specific Requirements

a. **Create a table that outlines all your functional requirements. The columns for the table should be ID, Description, Priority and additional notes**

b. **Create a table that outlines all your non-functional requirements. The columns for the table should be ID, Description, Priority and additional notes**

c. **Create a table that outlines all your external interface requirements. The columns for the table should be ID, Description, Priority and additional notes**

d. **Create a table that outlines all your technical requirements. The columns for the table should be ID, Description, Priority and additional notes**

e. **System features**

f. **Include UI Mock-ups (Initial sketches of key screens and/or interfaces)**

g. **Current challenges or pain points as well as mitigation strategies**

a. **Functional requirements**

ID	Description	Priority	Additional Notes
FR-01	The system shall generate Safe Job Procedures (SJPs) using an OpenAI API call, triggered automatically after payment.	High	Core pipeline exists; prompt and output structuring must be changed
FR-02	The AI-generated SJP must be formatted into a structured, sectioned Word template (using python-docx) with distinct areas for "Task Steps", "Potential Hazards", "Required PPE", and "Emergency Procedures".	High	
FR-03	An administrator shall review and approve the structured SJP before the customer can download it.	Medium	Admin dashboard exists; review/approval workflow may need hardening.
FR-04	Approved documents (manual, SJP) shall be downloadable as editable DOCX and PDF.	High	Existing download endpoints; ensure they serve the new template.
FR-05	The system shall provide a Hazard Register: after a valid business/NAICS code is	High	

	submitted, it must display a list of all relevant hazards and their control measures.		
FR-06	**The system shall generate a Training Matrix based on the Hazard Register, listing required training, certifications, and renewal periods for each hazard/job.**	**High**	
FR-07	**The user interface shall be visually redesigned with a new colour palette, typography, and component styling, matching sponsor-provided design examples.**	**Medium**	
FR-08	**The application shall be deployed to a cloud-based test environment with a public URL for sponsor review.**	**Very High**	
FR-09	**An administrator shall be able to manage the library of NAICS codes, hazards, and training mappings through the admin dashboard.**	**Low**	**Supports long-term maintainability**
FR-10	**The system shall support a subscription-based payment model for recurring legislative updates and ongoing document access.**	**Low**	**Architecture should support future transition; current MVP uses one-time payment.**
FR-11	**The admin dashboard shall display analytics: document generation trends, approval rates, revenue summary.**	**Low**	
FR-12	**Multi-province support shall be architected so that adding a province (beyond Ontario) requires only data/content changes, not code rewrites.**	**Low**	
FR-13	**The customer-facing flow (wizard, document viewing) shall be fully responsive and usable on mobile devices.**	**Medium**	

b. Non-functional requirements

ID	Description	Priority	Additional Notes
NFR-01	All AI-generated SJPs must be inaccessible to customers until an admin explicitly approves them.	Low	Prevent inaccurate AI content.
NFR-02	The test deployment must be accessible via public URL.	Very High	For sponsor to review.
NFR-03	The SJP generation and formatting process (excluding LLM response time) shall add no more than 5 seconds of overhead.	Low	Must be validated after template engine changes.
NFR-04	The Hazard Register and Training Matrix features shall be reachable within 3 clicks from the main admin order dashboard.	Very Low	
NFR-05	Hazard and training data shall be stored in database tables, not hardcoded in application logic, to allow future updates without code changes.	Medium	
NFR-06	The UI shall aim for WCAG 2.1 Level AA compliance (colour contrast, keyboard navigation, screen reader labels).	Low	To ensure web accessibility standards.

c. External interface requirements

ID	Description	Priority	Additional Notes
EI-01	Customer-facing web interface (React SPA) that collects company data and displays generated documents.	Medium	Will undergo UI re-theme and mobile optimisation.
EI-02	Admin Dashboard web interface for order review, content management, and analytics.	Low	
EI-03	python-docx library for assembling the structured SJP template and exporting DOCX.	Medium	

EI-04	Cloud hosting platform providing compute and managed database services.	Medium	Needed for test deployment; must support environment variables.
EI-05	Managed cloud database (MySQL-compatible) replacing the local Docker container in the test environment.	Medium	

d. Technical requirements

ID	Description	Priority	Additional Notes
TR-01	Two new database tables shall be created: hazards and training_requirements	Medium	
TR-02	The SJP template engine shall use the existing python-docx library to map LLM output into a pre-defined Word template.	High	Post-processing layer to parse AI text into sections.
TR-03	A test environment shall be deployed using a cloud platform with a managed MySQL database, Dockerised backend, and static frontend hosting.	Very High	
TR-04	Environment-specific configuration files (development, staging, production) shall be clearly separated, with staging configured for the public test URL.	Low	
TR-05	Automated end-to-end tests shall cover the critical flows: order creation, SJP generation & approval, and Hazard Register display.	Medium	
TR-06	Frontend API base URLs must be configurable via environment variables, eliminating the current hardcoded values.	Medium	Fixes a known limitation documented by the previous team.

e. System features

Feature 1: Customer Purchase Wizard & Payment
A business owner visits the site, views the three plan tiers (Basic, Comprehensive, Industry-Specific) and selects one. A multi-step wizard collects company name, province, and industry/NAICS code. The customer then completes payment via Stripe Checkout. Upon successful payment, the backend triggers the SJP generation pipeline.

Feature 2: Structured SJP Generation & Admin Approval
After payment, the system sends the industry code and province to the OpenAI API. The raw AI output is fed into a post-processing layer that parses the text and inserts it into a pre-built Word template (python-docx) with clearly separated sections: Task Steps, Potential Hazards, Required PPE, Emergency Procedures. The formatted document is held in a "pending review" state. An administrator reviews the document in the admin dashboard, makes edits if necessary, and approves it. Only then can the customer download the structured SJP as a DOCX or PDF.

Feature 3: Hazard Register & Training Matrix
Using the same NAICS code that identifies the business's operations, the system queries a new hazards database table and displays a clear list of every associated hazard along with its recommended controls. From this register, the system automatically generates a Training Matrix: a table showing, for each hazard, the required training, competency criteria, and renewal period. Both the Hazard Register and Training Matrix are accessible from the admin dashboard (and optionally from the customer's document suite).

Feature 4: Admin Dashboard & Content Management
Administrators can review orders, manage customers, approve pending SJPs, and view email logs, capabilities already provided by the previous team. The dashboard is extended to include:

Management of NAICS codes, hazards, and training mappings (so that content can be updated without developer intervention).

Analytics panels showing document generation trends, approval rates, and revenue summaries.

Feature 5: Multi-Province Scalability (Architectural Readiness)
The data model and prompt logic are designed so that adding a new province (e.g., British Columbia) involves adding province-specific legislative content and prompt modifiers, not rewriting the application. Ontario is the only active province in the MVP.

Feature 6: UI/UX Modernisation
The entire frontend receives a visual overhaul: a new colour palette, professional typography, and refined component styling, all implemented by reconfiguring the existing Tailwind theme. The changes are driven by design examples provided by the sponsor.

Feature 7: Cloud Test Deployment & Sponsor Review
The application is deployed to a public cloud platform with a managed MySQL database. The sponsor accesses the live URL, walks through the entire customer and admin flow, and provides feedback that drives the final iteration.

Feature 8: Subscription Model (Future Roadmap)
While the current MVP relies on one-time purchases, the database schema and Stripe integration are designed to accommodate recurring subscriptions. In a later phase, customers will be able to subscribe for continuous updates to their safety documents whenever legislation changes.

f. UI Moc-ups

- Images generated and edited using lucidchart.com

Figure 1: Structured SJP Output ("The Form")

- **Header:** "Safe Job Procedure: [Task Name]"
- **Main content is a table, not paragraphs**
- **Columns:** Step #, Job Step Description, Potential Hazards, Controls/PPE, Emergency Action.
- **Each row represents one procedural step.**
- **"Download as Word" and "Download as PDF" buttons at the top right.**

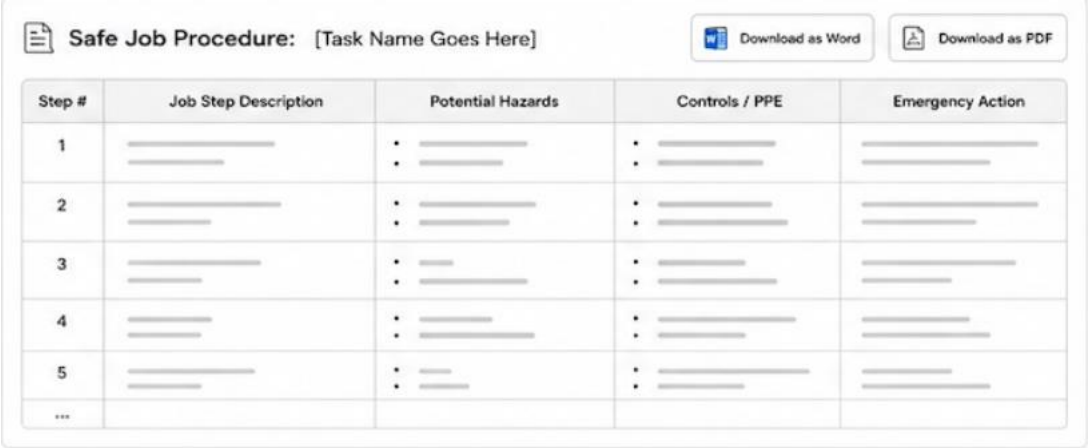


Figure 2: Hazard Register Page

- **Search bar labelled "Enter NAICS Code or Job Role:"**
- **Results area:** each hazard is a card/heading (e.g., "Working at heights"), underneath a bullet list of control measures.
- **Example: Hazard**: Electrical exposure → Controls: Lock-out/tag-out, insulated gloves, GFCI.

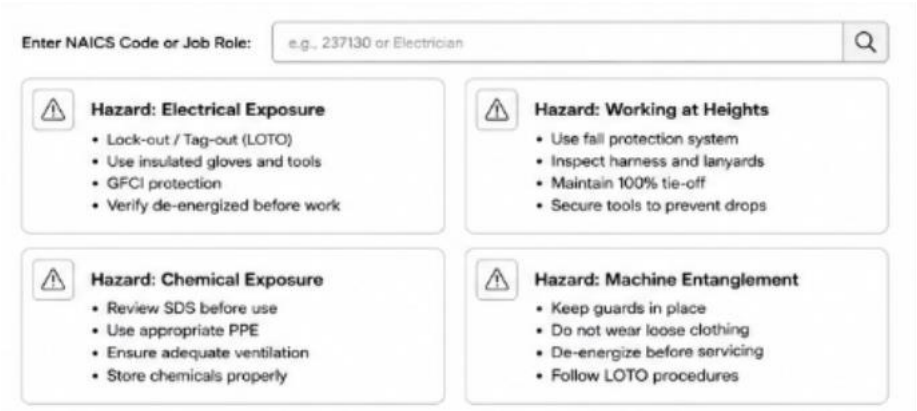


Figure 3: Training Matrix

- **Adjacent to or a separate tab from the Hazard Register.**
- **Clean table:**
  - **Job Role / Hazard | Required Training | Competency Evaluation | Renewal Period**
  - **Rows: "Working at Heights | Fall Protection Certification | Practical exam & written test | 3 years"**

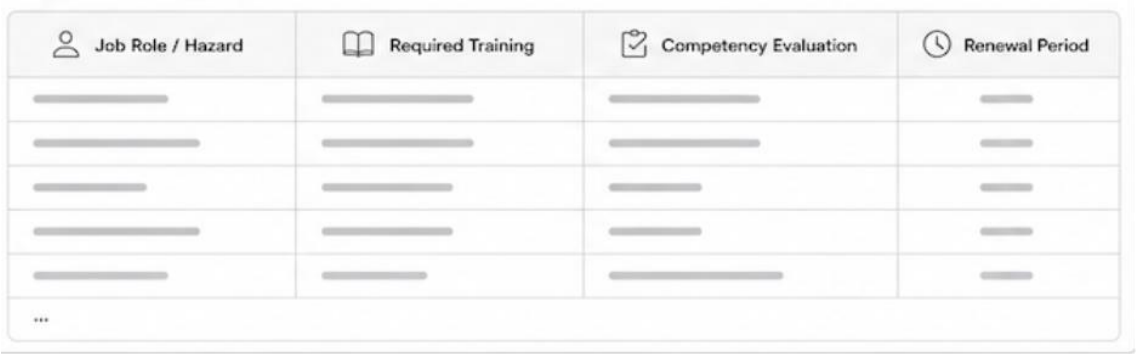
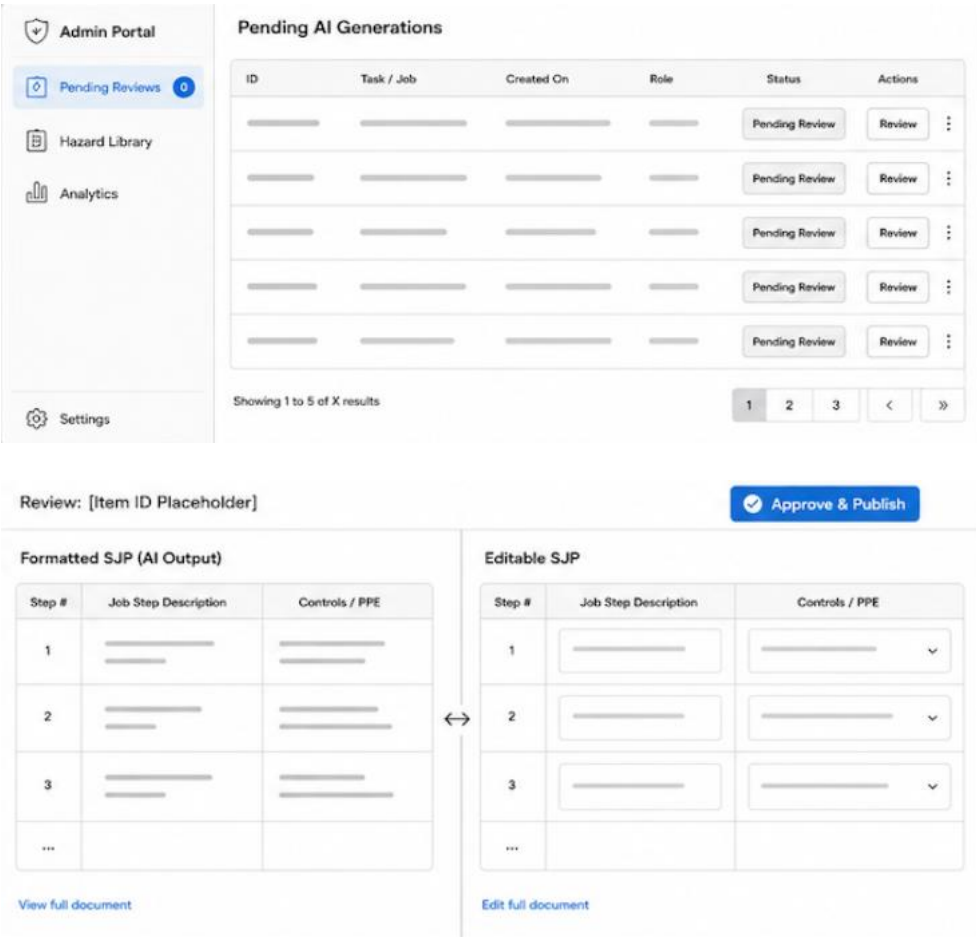


Figure 4: Admin Review Dashboard

- **Left sidebar:** Pending Reviews, Hazard Library, Analytics.
- **Main area: list of recent AI generations awaiting approval.**
- **Clicking "Review" opens a split view:** formatted SJP on the left, editable version on the right, with "Approve & Publish" button.



Navigation Flow (simple diagram):

Landing → Plan Selection → Company Info Wizard → Stripe Payment → (Backend AI +

Admin Review) → Structured SJP View / Hazard Register / Training Matrix → Download.

g. Current challenges and mitigation strategies

Challenges	Mitigation Strategies
Unpredictable LLM output: The AI may not always return content that fits the new structured template.	**Implement a deterministic post-processing layer (regex, keyword splitting) to parse and sanitise LLM output before inserting into the python-docx template. Include a user-visible fallback: "Content generation failed, please retry."**
Unstructured SJP format: Currently a continuous block of text, frustrating the sponsor.	**Define the Word template structure with Jennifer before coding. Modify AI prompts to request sectioned content. Short feedback loop with sponsor on sample output.**
Lack of hazard data: No existing hazard/training data; previous team left none. Scope to 5–10 common NAICS codes for the demo.	**Source starter data from Ontario's Ministry of Labour or open OHS databases. Seed the database with a migration script. Acknowledge in the report that an admin interface for adding more is future scope.**
Scope creep risk: Jennifer said there's "more stuff she would like" but doesn't want to overdo it.	**All low priority items go into a Future Backlog appendix. Sign-off on the completed Must-haves before adding any extras.**
Integration testing gaps: Past projects found late-stage bugs in end-to-end flows.	**Write automated E2E tests for the critical path: order creation - AI generation - template formatting - Hazard Register. Run on every pull request.**
Mobile experience may be sub-par, yet on-site business owners often use phones.	**Review every wizard screen and document preview at 375px width. Use Tailwind's responsive utilities to fix breakpoints. Prioritise customer-facing flows; admin dashboard can remain desktop-first initially.**

3. System Features

3.1 Front End:

Area	Choice
Build	Vite 5 (`@vitejs/plugin-react-swc`)
UI	React 18 + TypeScript (lax)
Router	`react-router-dom` v6
Server state	`@tanstack/react-query` v5
Forms	`react-hook-form` + `zod`
Styling	Tailwind 3 + CSS variables
Animation	`framer-motion`
Toasts	`sonner`
Icons	`lucide-react`
Testing	Vitest + jsdom + Testing Library

3.2 Back End:

Layer	Technology
Web framework	FastAPI (Python 3.11)
Data	MySQL 8.0 via SQLAlchemy 2.x + Alembic migrations
Validation	Pydantic v2
Documents	`python-docx` + Jinja2 DOCX templates in `templates/documents/`
Email	SMTP (Gmail in dev) with Jinja2 HTML templates in `templates/emails/`
Payments	Stripe Checkout (test mode in dev), webhook-driven status transitions | Handle payments
AI	OpenAI API for SJP content generation (`gpt-5-mini` by default)
Dev networking	ngrok tunnel so Stripe webhooks reach the local machine

| Container runtime | Docker + Docker Compose |

These technologies had been decided by the previous team. To not alter the current project scope, we are maintaining them and keeping on with the development.

4. Project Management Software:

Software| Usage

Jira| Define Sprints, Tasks Board and Timeline

SharePoint | Documentation

GitHub | Project Repositories and Development

Gmail | Communication with sponsor

5. Appendices

- **Glossary**

Term/Acronym	Type	Definition
AI (Artificial Intelligence)	Acronym/Technology	Technology that simulates human intelligence processes by machines, utilized within this system via the OpenAI API to dynamically draft safety text.
Alembic	Tech Stack	Lightweight database migration tool used with SQLAlchemy and Python.
API (Application Programming Interface)	Technical	A set of computing protocols allowing different software applications to communicate. It is used to connect the OHS Remote backend with Stripe and OpenAI.
CRUD (Create, Read, Update, and Delete)	Technical	CRUD is the four basic functions of persistent database storage utilized by administrators to

		manage NAICS system records.
DOCX	Technical	A Microsoft Word XML based editable file format. It is the primary template layout used for exporting generated safety manuals and procedures.
E2E (End to End) Testing	Technical	An automated testing methodology that checks the entire application flow from start to finish.
FastAPI	Tech Stack	A modern, high-performance Python 3.11 web framework used to construct the core backend API layer of the application.
Functional Requirement (FR)	Technical	A formal specification defining a distinct behavior, system feature, or action that the software application must execute.
GitHub	Project Management	A cloud based hosting service and repository platform utilized by the development team for version control.
Gmail	Project Management	The communication software used to handle primary digital correspondence and updates with the project sponsor.
gpt-5-mini	Tech Stack	The default large language model provided by OpenAI, used by the application to dynamically interpret context and draft core safety documents.
Hazard Register	System Feature	A core system component that queries a new hazards database

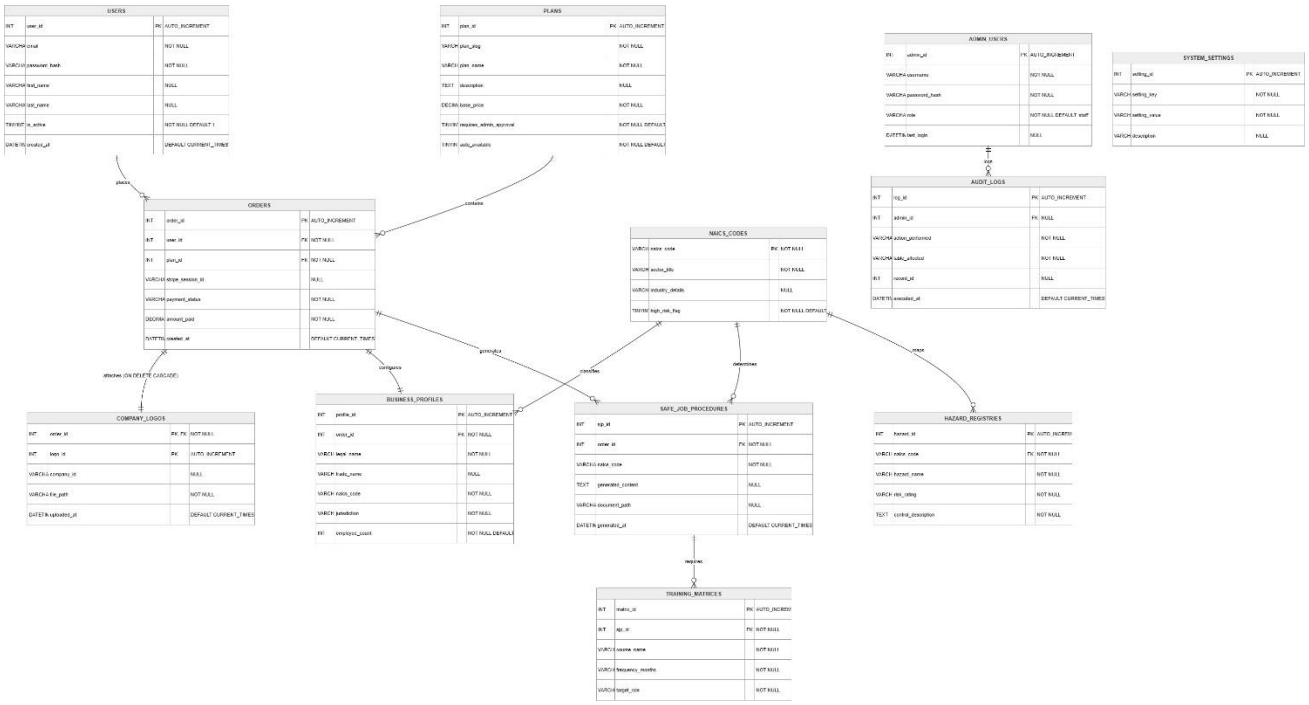
		table to display a clear list of all relevant occupational hazards matching a submitted business NAICS code.
Jira	Project Management	The project management software tool utilized by the development team to establish Sprints, host interactive task boards, and organize project timelines.
LLM (Large Language Model)	Technical	A type of deep learning artificial intelligence trained on vast text datasets, capable of understanding and generating human like textual outputs.
Lucidchart	Project Management	A cloud based diagramming and visual collaboration software used by the team to design and edit the system's UI Mockups.
MySQL 8.0	Tech Stack	An open source relational database management system utilized by the application to store persistent tables.
NAICS (North American Industry Classification System)	Domain / Acronym	A standard 6-digit economic classification code used across North America to identify a company's primary operational industry, which drives the safety generation criteria.
NFR (Non-Functional Requirement)	Technical	A formal statement defining quality attributes, performance constraints, or operational benchmarks of the system.

ngrok	Tech Stack	A cross platform dev networking tool that creates secure tunnels to expose a local development server to the internet, allowing external Stripe webhooks to reach local environments safety.
OpenAI	Tech Stack / External	The external cloud-based generative intelligence engine used by the backend application layer to generate relevant safe task steps.
PDF (Portable Document Format)	Technical	A universal file format used to display documents in an electronic form independent of the software, or operating system used to view them.
PPE (Personal Protective Equipment)	Domain	Specialized clothing, gear, or equipment worn by employees for protection against workplace hazards.
python-docx	Tech Stack	A Python library used within backend layer to map, parse, and format AI text into a predefined structured Word template.
React SPA	Tech Stack	A JavaScript frontend library that handles UI rendering directly in the browser to deliver a highly responsive, multi-step customer wizard.
SharePoint	Project Management	A web-based collaborative document storage platform provided by the institution to host, write, and maintain the

		group's centralized SRS report files.
SJP (Safe Job Procedure)	Domain/Acronym	A detailed, structured safety document mapping out a job description, step-by-step instructions, potential hazards, required PPE, and critical emergency actions.
SQLAlchemy 2.x	Tech Stack	An open-source SQL toolkit providing full corporate grade database access patterns for Python applications.
Stripe Checkout	Tech Stack / External	A secure, third-party payment processing platform used for secure transaction data before generating documents.
Tailwind 3	Tech Stack	A CSS styling framework configured alongside CSS variables for application typography, color themes, and responsive design breakpoints.
Training Matrix	System Feature	A core system grid that automatically cross references the Hazard Register to display required training courses, and renewal durations per hazard.
Vite 5	Tech Stack	A modern, highly optimized frontend build tool utilized to compile, bundle, and serve the React and TypeScript client application codebase.

WCAG 2.1 Level AA	Regulation / Standard	A set of globally recognized legal web accessibility standards focused on ensuring web user interfaces are entirely usable for individuals with sensory or physical disabilities.

- **Analysis model**
  Entity-Relationship Diagram



Note: ERD showing relationships between the tables

- **Supporting information**
  - Regulatory and Legal Frameworks
    1. North American Industry Classification System (NAICS): NAICS serves as the system's foundational classification data engine. OHS Remote maps a business's NAICS code to determine its industry sector, identify corresponding sector specific workplace hazards, and evaluate industry risk baselines.
    2. Ontario Occupational Health and Safety Act (OHSA): The Ontario OHSA mandates that employers provide information, instruction, and supervision to a worker to protect their health and safety.

The AI generated Safe Job Procedure handbooks enable businesses to fulfill this duty.

- Safety Theories

  The main focus of OHS Remote is the generated SJPs, corporate safety policies, operational rules, and structured Training Matrices. This helps employees to be safe from any type of workplace hazards.

- Architectural Design

  1. Security Separation of Entities: To prevent Public to admin authorization, (ADMIN_USERS, AUDIT-LOGS, SYSTEM_SETTINGS) are separated from core workflow.
  2. Stripe Tokens: To prevent any type of sensitive cardholder data loss, the application utilizes Stripe Checkout. All the transaction related data are stored in Stripe's secure infrastructure.

- References

  1. https://stripe.com/blog
  2. https://ngrok.com/docs/start